Final Project - Event-Driven Serverless Order Management System

High Level System Requirements

You are building a cloud-based Order Management System for an organization.

The system allows managing orders through REST APIs and must react automatically to important system events.

The system will allow managing notifications through REST APIs.

The system will keep a backup of deleted orders and will also support a summary report upon request.

The system will include a user interface, either a web application built with HTML or a Python-based menu application.

The entire solution must be serverless, event driven, and scalable.

Core Functional Requirements

Order Management (55 points)

1. The system will use one table named orders and will store the following details:
    - Order ID
    - Creation date
    - Price
    - Order description
    - Last modified date

    The table should be designed with proper primary and sort keys.

    The table must include a primary key that uniquely identifies each order. You may choose the key type, but it must be justified.

2. The system will expose the next REST APIs using API Gateway for the *orders* table:
    - Create order
    - Get all orders, sorted by creation date
    - Get a specific order
    - Update order details
    - Delete order

3. All orders data must be stored persistently in the backend

Notification Subscription (10 points)

1. The system will expose the following REST APIs for supporting notifications:
    - Register an email address to receive system notifications when an order is deleted.
    - Unsubscribe from notifications.
2. Registered users must receive notifications automatically, without any manual action

Order Deletion Process, Notification (10 points)

1. Once an order is deleted, an asynchronous notification to subscribed users by email will be executed, with the deleted order details.
2. Deleting an order must not wait for notifications. This process must run asynchronously.
    - The asynchronous notification of the deleted order details will not block nor delay in any way any other operation related to the order deletion process.
3. From the user perspective:
    - Deleting an order is a single action. The system responds immediately once order is deleted and does not wait for the notification to take place.
    - Notifications arrive automatically and do not block nor delay the response to the user.

Order Deletion Process, Backup in object storage (5 points)

1. Once an order is deleted, an asynchronous action will take place to keep the order details as a TXT file in an object storage.
2. Deleting an order must not wait for backup. This process must run asynchronously.
    - The creation of the backup data for the deleted orders will not block in any way any other operation related to the order deletion process.

Order Deletion Process, PDF Summary download (5 points)

1. The system will expose an API to support downloading a PDF that will include information for all deleted orders.
2. Once the API is called, it will go over all TXT files in object storage and will create a summary PDF with all the information and will return a URL to this PDF file. PDF should be kept in object storage so the user can download it.
3. The API must return the URL in the API response body so the user can download the PDF (Do not print in logs, return it so client can download it).

Freestyle enhancement (5 points)

Using one of the AWS services you were exposed to in the students' lecture or another service supported in the AWS Learner Lab, enhance the system with additional functionality using your selected AWS service. You are free to select the service and functionality. The AWS service should be implemented, and the client should be enhanced to support it. In any case, it should be clear which service was added, what functionality it provides and should be clearly visible from the UI.

The enhancement should add clear value and must be demonstrable.

Client Requirements (10 points)

The system must provide a client interface that allows users to perform all required actions by calling the REST APIs.

The client should only call APIs and display results. All business logic must be implemented in AWS services.

You may choose one of the following two implementation options.

Both options are considered equally valid (choose only one).

Option 1 – Web-Based Client

1. Implement a simple web client using: HTML, CSS and JavaScript.
2. The web application must demonstrate real calls to the backend APIs.

Option 2 – Python-Based Client

1. Implement a Python menu.
2. The Python program must support all required operations.
3. Make sure the client code is running smoothly. For any external dependencies (i.e. *pip install X*), make sure to install it from the python script or add a batch file that does all the work.

Common Requirements (Both Options)

- The client must:
    - Communicate with the backend using HTTP requests
    - Support all required operations, i.e., all operations which are called by an API, with the results of the operation.
    - Make sure to always display the result returned from the backend.
- The client must not contain any backend logic.
- The use of Generative AI tools to assist in creating the client is explicitly allowed and recommended.

Deliverables

Submission file

All the below deliverables, must be submitted in **1 WORD document**.

AWS Diagram

Submit an AWS diagram with all the services you use. Alongside the diagram, provide a clear, short, simple explanation about the diagram and how the services are integrated.

AWS Setup per service

Populate the next table:

AWS Service	Why did you choose this service? Explain the relation between the service and the functionality you are aiming to achieve by using it.	CLI Command to see the service details.

Note: The CLI Command must run successfully. Each service will be verified inside your AWS Learner lab. Please make sure names are correct, details are accurate. Please make sure to include all services, API, etc... used as part of the solution.

CLI commands must be complete and runnable without modification.

APIs List

Populate the following table:

API Name	HTTP Method (GET, POST, etc...)	API URL	Sample Input (For GET, URL with parameters, for POST, sample of body, etc...)	Sample output

Client URL / Code

Web Application

If you created a **web application**, provide the URL to the web application.

Make sure it's accessible and working as the project will be tested by the web page. Please host it on AWS Amplify.

Python Program

If you created a **Python** program, provide the .py file or files.

Make sure it passes compilation and runs smoothly as the project will be tested by the program.

List of tested flows

Provide a clear list of the flows that you tested, those should cover all the functionality.

For each tested flow, include screenshots and short explanations showing the steps and the results.

Note: Each of the above examples will be executed. All should work smoothly.

Freestyle enhancement explanation

Provide a clear explanation about the freestyle enhancement you implemented. Explain the reason you selected it, what is the functionality it adds to the system, and how to execute, verify it. Please make sure there is a clear way to operate the service / functionality.

Add a screenshot showing the enhancement.

The screenshot should clearly show the enhancement working in your UI.

Delete Order Code

Add the delete order python code (The Lambda Function).